

## **Resumen: Algoritmos Greedy (I)**

### **1. Introducción a los Algoritmos Greedy**

- Definición :  
Los algoritmos Greedy (o Voraces) son técnicas de diseño que buscan soluciones óptimas aplicando decisiones locales en cada paso, con la esperanza de alcanzar un óptimo global.
  - Toman decisiones basadas en una regla sencilla que maximiza/minimiza un criterio local.
  - Iteran aplicando esta regla hasta completar la solución.
- Filosofía :
  - Se comparan con el consecuencialismo/utilitarismo (el fin justifica los medios) frente al deontologismo (reglas absolutas).
  - No siempre garantizan el óptimo global, pero suelen ser rápidos e intuitivos.

### **2. Características Clave**

- Ventajas :
  - Son fáciles de entender e implementar.
  - Suelen ser rápidos y eficientes.
  - Pueden ofrecer buenas aproximaciones para problemas complejos.
- Desventajas :
  - No siempre garantizan el resultado óptimo.
  - Demostrar que un algoritmo Greedy es correcto puede ser difícil.

Importante : Un algoritmo Greedy debe tener una regla clara para seleccionar un óptimo local en cada paso. Si no se puede definir esta regla, probablemente no sea un algoritmo Greedy.

### **3. Ejemplos Clásicos de Algoritmos Greedy**

- Dijkstra : Encuentra el camino más corto desde un nodo inicial a todos los demás en un grafo.
  - Regla Greedy: Elige el nodo no visitado con la menor distancia acumulada.
- Prim : Construye un Árbol de Expansión Mínima (MST).
  - Regla Greedy: Agrega la arista de menor peso que conecta un nodo visitado con uno no visitado.
- Kruskal : También construye un MST.
  - Regla Greedy: Ordena las aristas por peso y las agrega si no forman ciclos.

#### **4. Problemas Avanzados Resueltos con Greedy**

##### **Problema I: Scheduling (Planificación de Charlas)**

- Objetivo : Maximizar el número de charlas en una sala, dadas sus horas de inicio y fin.
- Solución Greedy :
  - Ordenar las charlas por hora de finalización.
  - Seleccionar iterativamente la próxima charla que no se superponga con la última seleccionada.
- Complejidad :  $O(n \log n)$  debido al ordenamiento.

##### **Problema II: Árboles de Huffman**

- Objetivo : Comprimir texto utilizando códigos binarios de longitud variable basados en la frecuencia de caracteres.
- Solución Greedy :
  - Usar un heap (cola de prioridad) para combinar nodos con las menores frecuencias.
  - Construir un árbol binario donde los caracteres más frecuentes tienen códigos más cortos.
- Curiosidad : Huffman desarrolló este algoritmo como alternativa a rendir un examen final en su curso de Teoría de la Información.

##### **Problema III: Problema del Cambio**

- Objetivo : Devolver cambio usando la mínima cantidad de monedas/billetes.
- Solución Greedy :
  - Seleccionar siempre la moneda/billete de mayor denominación posible sin exceder el monto restante.
- Limitaciones :
  - Funciona bien con sistemas monetarios canónicos (como el nuestro), pero puede fallar en sistemas no estándar (e.g., [1, 5, 6, 9] para un cambio de 11).

##### **Problema IV: Compras con Inflación**

- Objetivo : Minimizar el costo total de comprar productos cuando los precios aumentan exponencialmente cada día.
- Solución Greedy :

- Comprar primero los productos con mayor tasa de inflación (mayor crecimiento diario).

### **Problema V: Carga de Combustible**

- Objetivo : Determinar las estaciones donde detenerse para minimizar el número de paradas durante un viaje largo.
- Solución Greedy :
  - Avanzar lo máximo posible antes de detenerse a cargar combustible.
  - Detenerse justo antes de que el tanque se agote.
- Complejidad :  $O(n)$ , donde  $n$  es el número de estaciones.

### **5. Reflexión Final**

- ¿Qué hace Greedy?
  - Aplica una sucesión de decisiones locales esperando alcanzar un óptimo global.
  - La clave está en identificar la regla Greedy específica para cada problema.
- Pregunta Fundamental :
  - ¿Por qué un algoritmo es Greedy?
  - Respuesta: Porque utiliza una regla simple para tomar decisiones locales en cada paso.

### **Cosas Más Importantes**

1. Regla Greedy : Todo algoritmo Greedy debe tener una regla clara para seleccionar un óptimo local.
2. Ejemplos Clásicos : Dijkstra, Prim, Kruskal.
3. Aplicaciones Prácticas :
  - Planificación de actividades (Scheduling).
  - Compresión de datos (Huffman).
  - Problema del cambio.
4. Limitaciones : No siempre garantizan el óptimo global, especialmente en problemas con restricciones no canónicas.

## Resumen: Algoritmos Greedy (II)

### 1. Repaso de Algoritmos Greedy

- Definición :
  - Aplicamos una regla sencilla para obtener un óptimo local en cada paso.
  - Iteramos con esta regla esperando alcanzar el óptimo global.
- Importante :
  - Un algoritmo es Greedy si tiene una regla clara basada en el estado actual que busca un óptimo local.
  - No siempre garantizan el óptimo global, pero suelen ser rápidos e intuitivos.

### 2. Problemas Avanzados Resueltos con Greedy

#### Problema I: Problema de la Mochila (Knapsack)

- Descripción :
  - Tenemos una mochila con capacidad  $W$  (peso o volumen).
  - Cada elemento tiene un peso  $w_i$  y un valor  $v_i$ .
  - Objetivo: Maximizar el valor total sin exceder la capacidad  $W$ .
- Enfoques Greedy :
  1. Ordenar por mayor valor : Falla en algunos casos (contraejemplo mostrado).
  2. Ordenar por menor peso : Funciona en algunos casos, pero no siempre.
  3. Ordenar por relación  $v_i/w_i$  : Funciona en muchos casos, pero no en todos (contraejemplo mostrado).
- Propuesta (Aproximación) :
  - Calcular tanto por  $v_i/w_i$  como por mayor valor y quedarse con la mejor solución.
  - Nota : Este problema es NP-Hard, y un algoritmo Greedy no siempre garantiza el óptimo.

#### Problema II: Problema de la Mochila Fraccionada

- Descripción :
  - Similar al problema anterior, pero ahora se permite tomar fracciones de los elementos.
  - Objetivo: Maximizar el valor total sin exceder la capacidad  $W$ .
- Solución Greedy :

- Ordenar los elementos por  $vi/wi$  y agregarlos completamente hasta que no quepan más.
- Si queda espacio, agregar una fracción del siguiente elemento.
- Complejidad :  $O(n \log n)$  debido al ordenamiento.
- Importante : En este caso, el algoritmo Greedy siempre da el óptimo .

### Problema III: Minimización de Latencia Máxima (Scheduling II)

- Descripción :
  - Tareas tienen un deadline  $di$  y una duración  $ti$ .
  - Si una tarea termina después de su deadline, se incurre en latencia  $li=fi-di$  (donde  $fi=si+ti$ ).
  - Objetivo: Minimizar la latencia máxima.
- Enfoques Greedy :
  1. Ordenar por menor duración  $ti$  : Falla en algunos casos.
  2. Ordenar por mayor duración  $ti$  : También falla.
  3. Ordenar por  $di-ti$  : No funciona en todos los casos.
  4. Ordenar por deadline  $di$  : Funciona .
- Demostración :
  - Se demuestra que cualquier schedule sin inversiones (ordenado por deadline) tiene la misma latencia máxima.
  - Además, cualquier schedule óptimo puede transformarse en uno sin inversiones sin aumentar la latencia máxima.

### Problema IV: Optimal Caching

- Descripción :
  - Tenemos una caché de tamaño  $k$  y una secuencia de pedidos de datos  $di$ .
  - Si  $di$  está en la caché, accedemos rápido. Si no, ocurre un cache miss y debemos traer el dato a la caché, posiblemente evictando otro.
  - Objetivo: Minimizar la cantidad de cache misses.
- Casos :
  1. Conocimiento futuro : Podemos usar algoritmos como Belady's Algorithm .
  2. Sin conocimiento futuro : Usar políticas como LRU (Least Recently Used) o FIFO .

### Problema V: Coloreo de Intervalos

- Descripción :
  - Tenemos  $n$  charlas con horarios de inicio y fin.
  - Disponemos de  $k$  salas.
  - Objetivo: Asignar las charlas a las salas minimizando el número de salas utilizadas.
- Solución Greedy :
  - Ordenar las charlas por hora de inicio.
  - Asignar cada charla a la primera sala disponible.
  - Si no hay salas disponibles, abrir una nueva sala.
  - Complejidad :  $O(n \log n)$  debido al ordenamiento.
  - Importante : Este algoritmo es óptimo.

### Problema VI: Iluminación de Submarinos

- Descripción :
  - Dada una matriz donde algunas celdas contienen submarinos, colocar faros para iluminarlos todos.
  - Cada faro ilumina su celda y todas las adyacentes (incluyendo diagonales).
- Solución Greedy :
  - Colocar faros en las celdas que maximicen la cobertura de submarinos no iluminados.
  - Repetir hasta que todos los submarinos estén iluminados.
  - Nota : Este algoritmo no siempre garantiza el óptimo.

### Problema VII: Generación de Grafos

- Descripción :
  - Dada una lista de grados  $[d_1, d_2, \dots, d_n]$ , construir un grafo simple sin bucles con esos grados.
- Solución Greedy :
  - Usar el algoritmo de Havel-Hakimi :
    1. Ordenar los grados en orden descendente.

2. Conectar el nodo con mayor grado a los siguientes nodos.
  3. Restar 1 al grado de los nodos conectados.
  4. Repetir hasta que todos los grados sean 0 o se detecte que no es posible.
- Complejidad :  $O(n^2)$  en el peor caso.

### Problema VIII: Asignación de Proyectos

- Descripción :
  - Una empresa tiene  $n$  empleados y  $k$  proyectos.
  - Cada proyecto requiere ciertos empleados y genera una ganancia.
  - Cada empleado puede trabajar en máximo 2 proyectos.
  - Objetivo: Maximizar la ganancia seleccionando proyectos.
- Solución Greedy :
  - Ordenar los proyectos por ganancia descendente.
  - Asignar proyectos mientras se respeten las restricciones de los empleados.
  - Nota : Este algoritmo no siempre garantiza el óptimo.

### Problema IX: Materias Compatibles

- Descripción :
  - Dada una lista de materias con varios cursos posibles, seleccionar un curso por materia sin que se solapen.
- Solución Greedy :
  - Usar la función **son\_compatibles(curso\_1, curso\_2)** para verificar compatibilidad.
  - Priorizar cursos que maximicen la compatibilidad con los ya seleccionados.

### Conclusiones Clave

1. Regla Greedy : Todo algoritmo Greedy debe tener una regla clara para seleccionar un óptimo local en cada paso.
2. Aplicaciones Prácticas :
  - Problema de la mochila (fraccionada y discreta).
  - Scheduling (minimización de latencia máxima).

- Optimal caching (LRU, FIFO).
- Coloreo de intervalos.
- Iluminación de submarinos.
- Generación de grafos (Havel-Hakimi).

3. Limitaciones :

- No siempre garantizan el óptimo global.
- Demostrar optimalidad puede ser difícil.

### **Cosas Más Importantes**

1. Mochila Fraccionada : Greedy es óptimo cuando se permite fraccionar elementos.
2. Minimización de Latencia Máxima : Ordenar por deadline es óptimo.
3. Coloreo de Intervalos : Greedy es óptimo al asignar salas.
4. Generación de Grafos : Algoritmo de Havel-Hakimi es clave.
5. Optimal Caching : LRU es una política común en sistemas reales.